

PySpark DataFrame

Commands Mastery

Complete Reference | Azure Data Engineer Edition

Read/Write

Transform

Aggregations

Joins

Window Fn

Optimise

All commands with syntax, examples and pro tips | 2025 Edition

1. Creating DataFrames

```
from pyspark.sql import SparkSession
from pyspark.sql.types import *
from pyspark.sql import functions
as F
spark = SparkSession.builder.appName("App").getOrCreate()
```

From Files

Command	Description	Example
<code>spark.read.csv()</code>	Read CSV file(s)	<code>spark.read.csv("path/*.csv", header=True, inferSchema=True)</code>
<code>spark.read.json()</code>	Read JSON file(s)	<code>spark.read.json("path/*.json", multiLine=True)</code>
<code>spark.read.parquet()</code>	Read Parquet files	<code>spark.read.parquet("path/*.parquet")</code>
<code>spark.read.orc()</code>	Read ORC files	<code>spark.read.orc("path/*.orc")</code>
<code>spark.read.format()</code>	Generic format reader	<code>spark.read.format("delta").load("path/")</code>
<code>spark.read.table()</code>	Read from catalog	<code>spark.read.table("catalog.schema.table")</code>
<code>spark.read.jdbc()</code>	Read from JDBC source	<code>spark.read.jdbc(url, "dbo.orders", properties=props)</code>

With Schema Definition

```
schema = StructType([ StructField("id", LongType(), nullable=False), StructField("name", StringType(), nullable=True), StructField("salary", DoubleType(), nullable=True), StructField("dob", DateType(), nullable=True), StructField("active", BooleanType(), nullable=True), StructField("tags", ArrayType(StringType()), nullable=True), StructField("meta", MapType(StringType(), StringType()), nullable=True), ])
df = spark.read.schema(schema).csv("path/data.csv", header=True)
```

From Python / Pandas

```
# From list of tuples
df = spark.createDataFrame([(1,"Alice",85.5),(2,"Bob",90.0)], ["id","name","score"])
# From list of dicts
df = spark.createDataFrame([{"id":1,"name":"Alice"}, {"id":2,"name":"Bob"}])
# From Pandas DataFrame
import pandas as pd
pdf = pd.DataFrame({"a":[1,2,3], "b":["x","y","z"]})
df = spark.createDataFrame(pdf)
# Empty DataFrame
df = spark.createDataFrame([], schema)
```

TIP Always define an explicit schema for production jobs. `inferSchema` reads the full file which is slow and may guess types incorrectly.

2. Inspecting DataFrames

Command	Description	Example / Output
<code>df.show(n)</code>	Print n rows to console	<code>df.show(10, truncate=False)</code>
<code>df.printSchema()</code>	Print schema tree	<code>df.printSchema()</code>
<code>df.schema</code>	StructType object	<code>df.schema['col'].dataType</code>
<code>df.dtypes</code>	List of (col, type) tuples	<code># [('id', 'bigint'), ('name', 'string')]</code>
<code>df.columns</code>	List of column names	<code># ['id', 'name', 'salary']</code>
<code>df.count()</code>	Total row count	<code>cnt = df.count()</code>

Command	Description	Example / Output
<code>df.describe()</code>	Summary stats (count/mean/std/min/max)	<code>df.describe('salary', 'age').show()</code>
<code>df.summary()</code>	Extended stats incl. percentiles	<code>df.summary('count', '25%', '75%').show()</code>
<code>df.explain()</code>	Print physical/logical plan	<code>df.explain(mode="extended")</code>
<code>df.rdd.getNumPartitions()</code>	Number of partitions	<code>n = df.rdd.getNumPartitions()</code>
<code>df.isEmpty()</code>	True if DataFrame has 0 rows	<code>if df.isEmpty(): ...</code>
<code>df.first()</code>	First Row object	<code>row = df.first(); row['name']</code>
<code>df.head(n)</code>	First n Row objects as list	<code>rows = df.head(5)</code>
<code>df.take(n)</code>	First n rows as list	<code>df.take(3)</code>
<code>df.tail(n)</code>	Last n rows as list	<code>df.tail(3)</code>
<code>df.collect()</code>	All rows as Python list (careful!)	<code>data = df.collect()</code>
<code>df.toPandas()</code>	Convert to Pandas DataFrame	<code>pdf = df.toPandas()</code>

NOTE Never call `df.collect()` or `df.toPandas()` on large DataFrames — it pulls ALL data to the Driver and causes `OutOfMemoryError`. Use only on small/filtered results.

3. Selecting & Renaming Columns

Command	Description	Example
<code>df.select()</code>	Select one or more columns	<code>df.select("id", "name", "salary")</code>
<code>df.select(F.col())</code>	Select with Column object	<code>df.select(F.col("salary")*1.1)</code>
<code>df.select(F.expr())</code>	Select with SQL expression	<code>df.select(F.expr("salary * 1.1 AS new_sal"))</code>
<code>df.selectExpr()</code>	Select with SQL string expressions	<code>df.selectExpr("id", "salary * 1.1 as adj_sal")</code>
<code>df.withColumn()</code>	Add or replace a column	<code>df.withColumn("tax", F.col("salary")*0.2)</code>
<code>df.withColumnRenamed()</code>	Rename a single column	<code>df.withColumnRenamed("emp_id", "id")</code>
<code>df.withColumnsRenamed()</code>	Rename multiple columns (dict)	<code>df.withColumnsRenamed({"a": "x", "b": "y"})</code>
<code>df.drop()</code>	Remove one or more columns	<code>df.drop("col1", "col2")</code>
<code>df.toDF(*cols)</code>	Rename all columns at once	<code>df.toDF("id", "name", "score")</code>
<code>df[col]</code>	Reference column by name	<code>df["salary"].alias("pay")</code>
<code>df.col()</code>	Same as <code>df[col]</code>	<code>df.col("name")</code>
<code>F.col()</code>	Free column reference	<code>F.col("dept").alias("department")</code>
<code>F.lit(value)</code>	Create literal column	<code>F.lit("active").alias("status")</code>
<code>F.expr(sql)</code>	SQL expression as column	<code>F.expr("CASE WHEN sal>50k THEN 'high' END")</code>

```
# Select with aliases and expressions df.select( F.col("emp_id").alias("id"), F.concat_ws(" ",
"first_name","last_name").alias("full_name"), (F.col("salary") * 12).alias("annual_salary"),
F.when(F.col("salary") > 80000, "Senior").otherwise("Junior").alias("grade"),
F.col("dept").alias("department"), )
```

4. Filtering Rows

Command	Description	Example
<code>df.filter(condition)</code>	Filter rows (alias: <code>df.where()</code>)	<code>df.filter(F.col("age") > 18)</code>
<code>df.where(condition)</code>	Same as <code>df.filter()</code>	<code>df.where(F.col("status") == "active")</code>
<code>&</code> (AND)	Combine conditions with AND	<code>(F.col("age")>18) & (F.col("active")==True)</code>
<code> </code> (OR)	Combine conditions with OR	<code>(F.col("dept")==IT") (F.col("dept")==HR")</code>
<code>~</code> (NOT)	Negate a condition	<code>~F.col("active")</code>
<code>F.col().isNull()</code>	Null check — is null	<code>df.filter(F.col("email").isNull())</code>
<code>F.col().isNotNull()</code>	Null check — not null	<code>df.filter(F.col("email").isNotNull())</code>
<code>F.col().isin()</code>	Value in a list	<code>df.filter(F.col("dept").isin("IT", "HR", "Finance"))</code>
<code>F.col().between()</code>	Range check (inclusive)	<code>df.filter(F.col("salary").between(40000 ,80000))</code>
<code>F.col().like()</code>	SQL LIKE pattern	<code>df.filter(F.col("name").like("A%"))</code>
<code>F.col().rlike()</code>	Regex match	<code>df.filter(F.col("email").rlike(r"@company\.com\$"))</code>
<code>F.col().startswith()</code>	Prefix match	<code>df.filter(F.col("code").startswith("TX"))</code>
<code>F.col().endswith()</code>	Suffix match	<code>df.filter(F.col("file").endswith(".csv"))</code>
<code>F.col().contains()</code>	Substring match	<code>df.filter(F.col("desc").contains("error"))</code>
<code>df.dropna()</code>	Drop rows with null values	<code>df.dropna(subset=["email", "phone"])</code>
<code>df.na.drop()</code>	Same as <code>dropna()</code>	<code>df.na.drop(how="any", subset=["id", "name"])</code>

```
# Complex multi-condition filter df.filter( (F.col("dept").isin("IT","Finance")) &
(F.col("salary").between(40000, 120000)) & (F.col("active") == True) & F.col("manager_id").isNotNull()
& ~F.col("name").like("%test%") ) # Filter with SQL string df.filter("dept = 'IT' AND salary > 50000")
# Filter using Spark SQL df.createOrReplaceTempView("emp") spark.sql("SELECT * FROM emp WHERE dept =
'IT' AND salary > 50000")
```

TIP Always put most selective filters first to minimise data early. Parquet/Delta will use predicate pushdown to skip row groups automatically.

5. Transformations & Column Operations

Adding & Modifying Columns

```
# Conditional column (CASE WHEN equivalent) df.withColumn("grade", F.when(F.col("score") >= 90, "A")
.when(F.col("score") >= 75, "B") .when(F.col("score") >= 60, "C") .otherwise("F")) # Type casting
df.withColumn("id", F.col("id").cast(LongType())) df.withColumn("salary",
F.col("salary").cast("double")) df.withColumn("date", F.col("date_str").cast(DateType())) # Null
filling and replacement df.fillna(0) # fill all nulls with 0 df.fillna({"salary":0, "dept":"Unknown"})
# fill per column df.na.fill({"salary":0, "dept":"Unknown"}) # same df.na.replace(["N/A", "null", ""],
None) # replace values with null df.replace({"M":"Male", "F":"Female"}, subset=["gender"])
```

String Functions

Function	Description	Example
F.upper() / F.lower()	Change case	F.upper(F.col("name"))
F.trim() / F.ltrim() / F.rtrim()	Strip whitespace	F.trim(F.col("name"))
F.length()	String length	F.length(F.col("name"))
F.substring(col, pos, len)	Extract substring (1-based)	F.substring("code", 1, 3)
F.split(col, pattern)	Split to array	F.split(F.col("tags"), ",")
F.concat()	Concatenate columns	F.concat(F.col("first"), F.lit(" "), F.col("last"))
F.concat_ws(sep, *cols)	Concat with separator	F.concat_ws("-", "yr", "mo", "dy")
F.regexp_replace()	Regex replace	F.regexp_replace("phone", "[^0-9]", "")
F.regexp_extract()	Regex capture group	F.regexp_extract("email", "(.+)@", 1)
F.instr(col, substr)	Find position of substr	F.instr(F.col("text"), "error")
F.lpad() / F.rpad()	Pad string to length	F.lpad(F.col("id"), 8, "0")
F.initcap()	Title-case each word	F.initcap(F.col("name"))
F.translate()	Char-by-char replacement	F.translate("col", "abc", "ABC")
F.format_string()	Printf-style formatting	F.format_string("%05d", F.col("id"))
F.locate(substr, col)	Position of first occurrence	F.locate("@", F.col("email"))

Date & Timestamp Functions

Function	Description	Example
F.current_date()	Today's date	F.withColumn("today", F.current_date())
F.current_timestamp()	Current datetime	F.withColumn("now", F.current_timestamp())
F.year/month/dayofmonth()	Extract date parts	F.year(F.col("ts")), F.month("ts")
F.hour/minute/second()	Extract time parts	F.hour(F.col("ts"))
F.date_add(col, days)	Add days to date	F.date_add(F.col("start_date"), 7)
F.date_sub(col, days)	Subtract days from date	F.date_sub(F.col("end_date"), 30)

Function	Description	Example
<code>F.datediff(end, start)</code>	Days between two dates	<code>F.datediff(F.col("end"), F.col("start"))</code>
<code>F.months_between()</code>	Months between two dates	<code>F.months_between("d2", "d1").cast("int")</code>
<code>F.add_months(col, n)</code>	Add n months	<code>F.add_months(F.col("date"), 3)</code>
<code>F.to_date(col, fmt)</code>	Parse string to Date	<code>F.to_date("date_str", "yyyy-MM-dd")</code>
<code>F.to_timestamp(col, fmt)</code>	Parse string to Timestamp	<code>F.to_timestamp("ts", "yyyy-MM-dd HH:mm:ss")</code>
<code>F.date_format(col, fmt)</code>	Format date to string	<code>F.date_format(F.col("ts"), "yyyy/MM/dd")</code>
<code>F.trunc(col, format)</code>	Truncate to period	<code>F.trunc(F.col("ts"), "month")</code>
<code>F.unix_timestamp()</code>	String to epoch seconds	<code>F.unix_timestamp("ts", "yyyy-MM-dd")</code>
<code>F.from_unixtime()</code>	Epoch seconds to string	<code>F.from_unixtime(F.col("epoch"))</code>
<code>F.from_utc_timestamp()</code>	UTC to local timezone	<code>F.from_utc_timestamp("ts", "Asia/Kolkata")</code>
<code>F.dayofweek(col)</code>	Day of week (1=Sunday)	<code>F.dayofweek(F.col("date"))</code>
<code>F.weekofyear(col)</code>	ISO week number	<code>F.weekofyear(F.col("date"))</code>
<code>F.last_day(col)</code>	Last day of month	<code>F.last_day(F.col("date"))</code>

Array & Map Functions

```
# Array functions
F.array("c1", "c2", "c3") # create array column
F.array_contains("arr_col", "value") # True if value in array
F.array_distinct("arr_col") # remove duplicates from array
F.array_union("a1", "a2") # union of two arrays
F.array_intersect("a1", "a2") # intersection
F.array_except("a1", "a2") # elements in a1 not in a2
F.array_sort("arr_col") # sort ascending
F.sort_array("arr_col", asc=False) # sort descending
F.array_size("arr_col") / F.size("arr_col") # number of elements
F.slice("arr_col", start=1, length=3) # subset of array (1-based)
F.element_at("arr_col", 1) # nth element (1-based)
F.flatten("nested_array_col") # flatten nested arrays
F.array_remove("arr_col", "val") # remove all occurrences
F.array_append("arr_col", F.lit("new_val")) # append element (Spark 3.4+)
F.arrays_zip("a1", "a2") # zip two arrays element-wise
F.arrays_overlap("a1", "a2") # True if any element shared
# Explode (array/map → multiple rows)
F.explode("arr_col") # one row per element (no nulls)
F.explode_outer("arr_col") # keep rows where array is null
F.posexplode("arr_col") # adds position index column
F.inline("array_of_structs") # explode array of structs → columns
# Map functions
F.create_map("k1", "v1", "k2", "v2") # create map column
F.map_from_arrays("keys_col", "vals_col") # create map from two arrays
F.map_keys("map_col") # extract keys as array
F.map_values("map_col") # extract values as array
F.map_contains_key("map_col", "key") # True if key exists
F.element_at("map_col", "key") # get value by key
```

6. Sorting, Deduplication & Sampling

Command	Description	Example
<code>df.orderBy()</code>	Sort rows (alias: <code>df.sort</code>)	<code>df.orderBy("salary", F.col("name").desc())</code>
<code>df.sort()</code>	Same as <code>orderBy()</code>	<code>df.sort(F.col("score").desc_nulls_last())</code>
<code>F.col().asc()</code>	Ascending sort order	<code>df.orderBy(F.col("date").asc())</code>
<code>F.col().desc()</code>	Descending sort order	<code>df.orderBy(F.col("salary").desc())</code>
<code>F.col().asc_nulls_first()</code>	Ascending, nulls first	<code>df.sort(F.col("v").asc_nulls_first())</code>
<code>F.col().desc_nulls_last()</code>	Descending, nulls last	<code>df.sort(F.col("v").desc_nulls_last())</code>
<code>df.distinct()</code>	Remove all duplicate rows	<code>df.distinct()</code>
<code>df.dropDuplicates(subset)</code>	Deduplicate by specific cols	<code>df.dropDuplicates(["id", "date"])</code>
<code>df.drop_duplicates()</code>	Alias for <code>dropDuplicates</code>	<code>df.drop_duplicates(["user_id"])</code>
<code>df.limit(n)</code>	Keep first n rows	<code>df.limit(1000)</code>
<code>df.sample(fraction)</code>	Random sample	<code>df.sample(fraction=0.1, seed=42)</code>
<code>df.sample(replace, frac, seed)</code>	With/without replacement	<code>df.sample(True, 0.05, 99)</code>
<code>df.sampleBy(col, fracs)</code>	Stratified sample by column	<code>df.sampleBy("dept", {"IT":0.5, "HR":1.0})</code>
<code>df.randomSplit(weights)</code>	Split into multiple DFs	<code>train, test = df.randomSplit([0.8,0.2], 42)</code>

7. Aggregations & GroupBy

```
# Standard groupBy aggregation
df.groupBy("dept", "year").agg(
  F.count("*").alias("headcount"),
  F.countDistinct("emp_id").alias("unique_emps"),
  F.sum("salary").alias("total_salary"),
  F.avg("salary").alias("avg_salary"),
  F.max("salary").alias("max_salary"),
  F.min("salary").alias("min_salary"),
  F.stddev("salary").alias("std_salary"),
  F.variance("salary").alias("var_salary"),
  F.percentile_approx("salary", 0.5).alias("median_salary"),
  F.percentile_approx("salary", [0.25, 0.75]).alias("quartiles"),
  F.first("manager_id", ignorenulls=True).alias("manager"),
  F.last("status").alias("latest_status"),
  F.collect_list("name").alias("names_list"),
  F.collect_set("skill").alias("unique_skills"),
  F.sum_distinct("bonus").alias("total_unique_bonus"),
)
```

Aggregation Functions Reference

Function	Description	Notes
<code>F.count('*')</code>	Count all rows incl. nulls	<code>F.count(col)</code> excludes nulls
<code>F.countDistinct(col)</code>	Count unique non-null values	Can pass multiple cols
<code>F.approx_count_distinct(col, rsd)</code>	Fast approx distinct count	<code>rsd=0.05</code> (5% error)
<code>F.sum(col)</code>	Sum of values	Ignores nulls

Function	Description	Notes
F.avg(col) / F.mean(col)	Mean average	Ignores nulls
F.max(col)	Maximum value	Works on strings & dates too
F.min(col)	Minimum value	Works on strings & dates too
F.stddev(col)	Sample standard deviation	F.stddev_pop() for population
F.variance(col)	Sample variance	F.var_pop() for population
F.skewness(col)	Statistical skewness	Useful for skew detection
F.kurtosis(col)	Statistical kurtosis	Distribution shape measure
F.corr(col1,col2)	Pearson correlation	Returns -1 to 1
F.covar_samp(c1,c2)	Sample covariance	F.covar_pop() for population
F.percentile_approx(col,p)	Approximate percentile	p can be list e.g. [0.25,0.5,0.75]
F.collect_list(col)	All values as array (with dups)	Order not guaranteed
F.collect_set(col)	Unique values as array	Order not guaranteed
F.first(col, ignorenulls)	First value in group	Set ignorenulls=True
F.last(col, ignorenulls)	Last value in group	Set ignorenulls=True
F.sum_distinct(col)	Sum of distinct values	Ignores duplicates
F.any_value(col)	Any value from group (Spark 3.4+)	Non-deterministic

Cube, Rollup & Pivot

```
# ROLLUP – subtotals along hierarchy df.rollup("region","dept").agg(F.sum("sales").alias("total")) #
Produces: (region,dept), (region,null), (null,null) # CUBE – all combinations of subtotals
df.cube("region","dept").agg(F.sum("sales").alias("total")) # Produces: (r,d), (r,null), (null,d),
(null,null) # PIVOT – rows to columns df.groupBy("dept").pivot("year",
[2022,2023,2024]).sum("revenue") # Produces: dept | 2022 | 2023 | 2024 # Global aggregation (no
groupBy) df.agg(F.sum("salary"), F.avg("salary"), F.count("*"))
```

8. Joins

Join Type	Description	Example
inner	Rows matching in both DFs	df1.join(df2, "id", "inner")
left / left_outer	All left + matching right	df1.join(df2, "id", "left")
right / right_outer	All right + matching left	df1.join(df2, "id", "right")
outer / full / full_outer	All rows from both sides	df1.join(df2, "id", "outer")
left_semi	Left rows where key exists in right	df1.join(df2, "id", "left_semi")
left_anti	Left rows where key NOT in right	df1.join(df2, "id", "left_anti")

Join Type	Description	Example
cross	Cartesian product (every combo)	<code>df1.crossJoin(df2)</code>

```
# Join on single column (drops duplicate key col) df_orders.join(df_customers, on="customer_id",
how="left") # Join on multiple columns df1.join(df2, on=["order_id","date"], how="inner") # Join with
expression (keeps both key columns) df1.join(df2, df1["id"] == df2["emp_id"], "inner") # Join with
column rename to avoid ambiguity df1.alias("a").join(df2.alias("b"), F.col("a.id") == F.col("b.id")) #
Broadcast join – for small lookup tables (fits in memory) from pyspark.sql.functions import broadcast
df_large.join(broadcast(df_small), on="product_id", how="left") # Semi-join: find orders that have a
matching customer df_orders.join(df_customers, "customer_id", "left_semi") # Anti-join: find orders
with NO matching customer df_orders.join(df_customers, "customer_id", "left_anti")
```

TIP Use `broadcast()` for any table under ~200 MB. Set `spark.sql.autoBroadcastJoinThreshold=50m`. Broadcast joins skip shuffle entirely.

NOTE Avoid joining on nullable columns without filtering nulls first — `null != null` in join conditions, causing unexpected row loss or duplicates.

9. Window Functions

```
from pyspark.sql.window import Window # Define window specifications w_dept =
Window.partitionBy("dept").orderBy("salary") w_dept_desc =
Window.partitionBy("dept").orderBy(F.col("salary").desc()) w_date =
Window.partitionBy("dept").orderBy("date") w_rows =
Window.partitionBy("dept").orderBy("salary").rowsBetween(-2, 0) w_range =
Window.partitionBy("dept").orderBy("salary").rangeBetween(-5000, 5000) w_unbounded =
Window.partitionBy("dept").orderBy("date") \ .rowsBetween(Window.unboundedPreceding,
Window.currentRow) w_no_order = Window.partitionBy("dept") # for agg without ordering
```

Function	Description	Example
<code>F.rank()</code>	Rank with gaps (1,2,2,4)	<code>F.rank().over(w_dept)</code>
<code>F.dense_rank()</code>	Rank without gaps (1,2,2,3)	<code>F.dense_rank().over(w_dept)</code>
<code>F.row_number()</code>	Unique sequential number	<code>F.row_number().over(w_dept_desc)</code>
<code>F.percent_rank()</code>	Relative rank 0.0 to 1.0	<code>F.percent_rank().over(w_dept)</code>
<code>F.cume_dist()</code>	Cumulative distribution 0 to 1	<code>F.cume_dist().over(w_dept)</code>
<code>F.ntile(n)</code>	Divide into n equal buckets	<code>F.ntile(4).over(w_dept)</code>
<code>F.lag(col, n, default)</code>	Value n rows before	<code>F.lag('salary',1,0).over(w_date)</code>
<code>F.lead(col, n, default)</code>	Value n rows after	<code>F.lead('salary',1,0).over(w_date)</code>
<code>F.first(col)</code>	First value in window	<code>F.first('salary').over(w_dept)</code>
<code>F.last(col)</code>	Last value in window	<code>F.last('salary').over(w_dept)</code>
<code>F.sum(col)</code>	Running / window sum	<code>F.sum('salary').over(w_rows)</code>
<code>F.avg(col)</code>	Running / window avg	<code>F.avg('salary').over(w_unbounded)</code>
<code>F.max(col)</code>	Running max in window	<code>F.max('salary').over(w_unbounded)</code>
<code>F.min(col)</code>	Running min in window	<code>F.min('salary').over(w_unbounded)</code>
<code>F.count(col)</code>	Running count in window	<code>F.count('*').over(w_unbounded)</code>

```
# Common patterns # Top-N per group (top 3 salaries per dept) df.withColumn("rnk",
F.row_number().over(w_dept_desc)) \ .filter(F.col("rnk") <= 3) # Running total per group sorted by
date df.withColumn("running_total", F.sum("revenue").over(Window.partitionBy("dept") .orderBy("date")
.rowsBetween(Window.unboundedPreceding, Window.currentRow))) # Month-over-month difference
df.withColumn("prev_month_rev",
F.lag("revenue",1,0).over(Window.partitionBy("region").orderBy("month"))) \ .withColumn("mom_change",
F.col("revenue") - F.col("prev_month_rev"))
```

10. Partitioning & Performance

Command	Description	Example
<code>df.repartition(n)</code>	Reshuffle to exactly n partitions	<code>df.repartition(200)</code>
<code>df.repartition(n, col)</code>	Partition by column value	<code>df.repartition(200,"country")</code>
<code>df.coalesce(n)</code>	Reduce partitions (no full shuffle)	<code>df.coalesce(10)</code>
<code>df.cache()</code>	Cache in memory+disk	<code>df.cache(); df.count()</code>
<code>df.persist(level)</code>	Cache with specific StorageLevel	<code>df.persist(StorageLevel.MEMORY_ONLY)</code>
<code>df.unpersist()</code>	Release cached data	<code>df.unpersist()</code>
<code>df.localCheckpoint()</code>	Truncate lineage (local)	<code>df.localCheckpoint()</code>
<code>df.checkpoint()</code>	Truncate lineage (HDFS/ADLS)	<code>df.checkpoint()</code>
<code>broadcast(df)</code>	Hint to broadcast join	<code>df_large.join(broadcast(df_small))</code>
<code>df.hint('skew', col)</code>	Skew join hint (Databricks)	<code>df.hint("skew", "user_id")</code>
<code>df.hint('coalesce', n)</code>	Optimizer coalesce hint	<code>df.hint("coalesce", 5)</code>
<code>df.hint('repartition', n)</code>	Optimizer repartition hint	<code>df.hint("repartition", 200)</code>

```
# Adaptive Query Execution (enable in Spark 3+) spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.adaptive.advisoryPartitionSizeInBytes", "128m")
spark.conf.set("spark.sql.shuffle.partitions", "200")
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "52428800") # 50MB # Check and view explain
plan df.explain() # default (simple) df.explain("extended") # logical + physical plans
df.explain("cost") # with cost estimates df.explain("codegen") # generated code
df.explain("formatted") # formatted physical plan
```

TIP Cache a DataFrame ONLY when you reuse it 2+ times in the same job. Always call `df.count()` after `df.cache()` to materialise the cache eagerly.

11. Reading & Writing Options

Read Options

```
# CSV options spark.read.csv("path/", header=True, # first row is header inferSchema=True, #
auto-detect types (slow – prefer explicit schema) sep="|", # delimiter quote="'", # quote character
escape="\\", # escape character nullValue="NULL", # string to treat as null nanValue="NaN", # string
to treat as NaN encoding="UTF-8", multiLine=True, # allow newlines in quoted fields
ignoreLeadingWhiteSpace=True, ignoreTrailingWhiteSpace=True, mode="DROPMALFORMED", # PERMISSIVE /
DROPMALFORMED / FAILFAST columnNameOfCorruptRecord="_corrupt_record", ) # Parquet / Delta options
spark.read.parquet("path/").select("only","needed","cols") # column pruning
spark.read.format("delta").option("versionAsOf", 5).load("path/")
spark.read.format("delta").option("timestampAsOf", "2024-06-01").load("path/")
spark.read.format("delta").option("readChangeFeed", "true") \
.option("startingVersion", 10).table("catalog.schema.table")
```

Write Options

```
# Write modes df.write.mode("overwrite") # replace all existing data df.write.mode("append") # add
rows to existing data df.write.mode("ignore") # skip if data already exists df.write.mode("error") #
raise error if data exists (default) # Write partitioned
df.write.mode("overwrite").partitionBy("year","month").parquet("path/") # Control file count
```

```
df.repartition(10).write.mode("overwrite").parquet("path/") # 10 files
df.coalesce(1).write.mode("overwrite").csv("path/") # single file # Parquet / Delta write options
df.write.mode("overwrite") \ .option("compression","snappy") \ .parquet("path/")
df.write.format("delta") \ .mode("overwrite") \ .option("overwriteSchema","true") \
.option("mergeSchema","true") \ .option("replaceWhere","year=2024") \ .partitionBy("year","month") \
.save("abfss://container@storage.dfs.core.windows.net/delta/orders/") # Save as managed table
df.write.mode("overwrite").saveAsTable("catalog.schema.table")
df.writeTo("catalog.schema.table").overwritePartitions() # Spark 3+
```

12. Null Handling & Data Cleaning

Command	Description	Example
<code>df.fillna(val)</code>	Replace null with value	<code>df.fillna({"salary":0, "name":"Unknown"})</code>
<code>df.na.fill()</code>	Same as <code>fillna()</code>	<code>df.na.fill(0, subset=["qty", "price"])</code>
<code>df.dropna()</code>	Drop rows with any/all null	<code>df.dropna(how="any", subset=["id", "email"])</code>
<code>df.na.drop()</code>	Same as <code>dropna()</code>	<code>df.na.drop(how="all")</code>
<code>df.replace()</code>	Replace specific values	<code>df.replace("N/A", None, subset=["col"])</code>
<code>df.na.replace()</code>	Same as <code>replace()</code>	<code>df.na.replace(["", "-"], None)</code>
<code>F.coalesce()</code>	Return first non-null column	<code>F.coalesce(F.col("a"), F.col("b"), F.lit(0))</code>
<code>F.isnull(col)</code>	Column boolean: is null?	<code>F.isnull(F.col("email"))</code>
<code>F.isnan(col)</code>	Column boolean: is NaN?	<code>F.isnan(F.col("score"))</code>
<code>F.nanvl(col, val)</code>	Replace NaN with value	<code>F.nanvl(F.col("score"), F.lit(0.0))</code>
<code>F.nullif(col, val)</code>	Return null if <code>col==val</code>	<code>F.nullif(F.col("dept"), F.lit("N/A"))</code>
<code>F.ifnull(col, val)</code>	Return <code>val</code> if <code>col</code> is null (SQL)	<code>F.expr("ifnull(salary, 0)")</code>
<code>F.nvl(col, val)</code>	Oracle-style null replace	<code>F.nvl(F.col("val"), F.lit(0))</code>

13. Schema & Catalog Operations

Command	Description	Example
<code>df.schema</code>	Get StructType schema	<code>schema = df.schema</code>
<code>df.printSchema()</code>	Print schema as tree	<code>df.printSchema()</code>
<code>df.dtypes</code>	List of (col, dtype) tuples	<code>df.dtypes</code>
<code>df.columns</code>	List of column names	<code>cols = df.columns</code>
<code>spark.catalog.listDatabases()</code>	List all databases	<code>spark.catalog.listDatabases()</code>
<code>spark.catalog.listTables(db)</code>	List tables in a database	<code>spark.catalog.listTables("mydb")</code>
<code>spark.catalog.listColumns(tbl)</code>	List columns of table	<code>spark.catalog.listColumns("orders")</code>

Command	Description	Example
<code>spark.catalog.tableExists()</code>	Check if table exists	<code>spark.catalog.tableExists("db.tbl")</code>
<code>spark.catalog.isCached()</code>	Check if table is cached	<code>spark.catalog.isCached("orders")</code>
<code>spark.catalog.cacheTable()</code>	Cache a table	<code>spark.catalog.cacheTable("orders")</code>
<code>spark.catalog.uncacheTable()</code>	Uncache a table	<code>spark.catalog.uncacheTable("orders")</code>
<code>spark.catalog.dropTempView()</code>	Drop temp view	<code>spark.catalog.dropTempView("v")</code>
<code>spark.catalog.refreshTable()</code>	Refresh metadata/cache	<code>spark.catalog.refreshTable("orders")</code>

```
# Register views df.createOrReplaceTempView("orders") # session-scoped
df.createOrReplaceGlobalTempView("orders_global") # cross-session # Access global temp views
spark.sql("SELECT * FROM global_temp.orders_global") # Useful SQL via spark.sql() spark.sql("SHOW
DATABASES") spark.sql("USE my_catalog.bronze") spark.sql("SHOW TABLES IN my_catalog.bronze")
spark.sql("DESCRIBE TABLE my_catalog.bronze.events") spark.sql("DESCRIBE EXTENDED
my_catalog.bronze.events") spark.sql("DESCRIBE DETAIL my_catalog.bronze.events") # Delta only
spark.sql("ANALYZE TABLE orders COMPUTE STATISTICS FOR ALL COLUMNS")
```

14. Miscellaneous Must-Know Commands

Command	Description	Example
<code>df.union(other)</code>	Stack two DFs (by position)	<code>df1.union(df2)</code>
<code>df.unionAll(other)</code>	Alias for union()	<code>df1.unionAll(df2)</code>
<code>df.unionByName(other)</code>	Stack by column name	<code>df1.unionByName(df2, allowMissingColumns=True)</code>
<code>df.intersect(other)</code>	Rows in both DFs	<code>df1.intersect(df2)</code>
<code>df.intersectAll(other)</code>	Intersect keeping duplicates	<code>df1.intersectAll(df2)</code>
<code>df.subtract(other)</code>	Rows in df1 not in df2	<code>df1.subtract(df2)</code>
<code>df.exceptAll(other)</code>	Subtract keeping duplicates	<code>df1.exceptAll(df2)</code>
<code>df.crossJoin(other)</code>	Cartesian product	<code>df1.crossJoin(df2)</code>
<code>df.withWatermark(col, delay)</code>	Watermark for streaming	<code>df.withWatermark("ts", "10 minutes")</code>
<code>df.transform(func)</code>	Apply a custom function to DF	<code>df.transform(my_clean_fn)</code>
<code>df.pipe(func)</code>	Pandas-style pipe (alias)	<code>df.pipe(add_audit_cols)</code>
<code>F.struct(*cols)</code>	Combine columns into struct	<code>F.struct("id", "name", "salary")</code>
<code>F.to_json(col)</code>	Convert struct/map to JSON string	<code>F.to_json(F.struct("*"))</code>
<code>F.from_json(col, schema)</code>	Parse JSON string to struct	<code>F.from_json("json_col", schema)</code>

